2. Entity a JPA anotácie
2.1 Štruktúra entitnej triedy
2.2 Generované kľúče
2.3 Identita entitných objektov
2.4 Synchronizácia stavu s databázou - Metódy entity managera
2.5 Generovanie entitnej triedy z tabuľky
2.6 JPQL dotazy
2.7 JPA controller
Cvičenia

2. Entity a JPA anotácie

V tejto časti sa detailnejšie oboznámime so štruktúrou entitných tried a funkcionalitou entity managera.

V novom projekte si najprv vygenerujeme entitnú triedu wizardom prostredia `netbeans` a ilustrujeme:

- generované kľúče
- persistence context a problematiku rovnosti entitných objektov
- metódy entity managera
- mapovanie rôznych dátových typov
- unikátne a nepovinné stĺpce

Potom si ukážeme generovanie entitnej triedy z databázovej tabuľky.
Nakoniec sa oboznámime s dotazovacím jazykom JPQL

2.1 Štruktúra entitnej triedy

IDE `netbeans` poskytuje viaceré nástroje na vytvorenie typickej entitnej triedy podľa konvencií ORM spĺňajúce požiadavky JPA. Nástroje na vytvorenie nájdeme v kontextovom menu projektu.
Nástroj `New/Other/Persistence Entity class...` - vytvorí základ entitnej triedy, ktorá:

- implementuje interface `Serializable`
- obsahuje len dátový člen `id` pre kľúčový stĺpec tabuľky – má anotáciu `@Id`,
- a implementuje metódy `equals` a `hash` - tak, že dva inštancie s rovnakým kľúčom predstavujú tú istú entitu (záznam) v relačnej databáze

Ďalšie dátové členy s prístupovými metódami môžeme do triedy pridať ručne.
V prostredí NetBeans môžeme použiť tiež nástroj `Add property…` v kontextovom menu editora.

V novom projekte vytvoríme nástrojmi prostredia NetBeans. entitnú triedu Person. Do triedy na ilustráciu pridáme dátové členy rôznych typov.

```
    private String name;
    private double salary;
    @Temporal(jakarta.persistence.TemporalType.DATE)
    private Date born;
    private boolean married;
```

Všimnite si:

- Anotáciu `@Temporal`, ktorá je povinná pre dátové členy typu `Date`.
  Určuje dátový typ stĺpca v databáze. Databázy totiž poskytujú rôzne dátové typy pre údaje o čase: `DATE, TIME, TIMESTAMP.`
- Názov prístupovej metódy `isMarried()`.

Podľa konvencií java-bean sa názov *getter* metódy pre dátový člen typu `boolean` tvorí pomocou prefixu `is` namiesto `get`.

2.2 Generované kľúče

V entitnej triede `Person` si všimnite, že kľúčový dátový člen `id` má okrem anotácie `@Id` aj anotáciu `@GeneratedValue`. Táto notácia zabezpečí, že unikátnu hodnotu kľúčového stĺpca vygeneruje samotná databáza pri vložení nového záznamu. Nie je teda potrebné inicializovať hodnotu kľúča v novo vytvorenom objekte.

V projekte si to ilustrujeme jednoduchou aplikáciou - `main` metóda. Metóda vytvorí inštanciu triedy `Person`, inicializuje hodnoty dátových členov, však hodnotu pre kľúč nezadá. Potom zavolá metódu `persist` na vytvorenie nového záznamu v databáze.

Po ukončení transakcie sa môžeme presvedčiť výpisom na obrazovku, že kľúčový dátový člen id obsahuje nenulovú hodnotu vygenerovanú databázou.

Všimnite si tiež aký databázový typ majú stĺpce `MARRIED` a `BORN`.

Implementáciu nájdete v projekte **p21a**.

2.3 Identita entitných objektov

Kostra entitnej triedy `Person` vytvorená v projekte **p21a** má už vlastnú implementáciu metód `hashCode` a `equals`.

```
    @Override
    public boolean equals(Object object) {
        if (!(object instanceof Person)) {
            return false;
        }
        Person other = (Person) object;
        if ((this.id == null && other.id != null) ||
            (this.id != null && !this.id.equals(other.id))) {
            return false;
        }
        return true;
    }
```

Ako vidíme, implementované sú tak, aby dva objekty boli považované za ekvivalentné (t. j. identické) práve vtedy, keď majú rovnaký kľúč, teda reprezentujú ten istý záznam v databáze.

Pripomeňme, že v relačných databázach nesmú existovať v jednej tabuľke dva záznamy s rovnakým primárnym kľúčom. Na rozdiel od toho, objektovo orientované programovanie umožňuje súčasnú existenciu viacerých inštancií entitnej triedy s rovnakou hodnotou kľúčového dátového člena, ostatné dátové členy môžu mať pri tom aj rôzne hodnoty.

Preto manažér entít môže manažovať t. j. synchronizovať s databázou len jednu z týchto inštancií. Nazývame ju preto manažovaná alebo pripojená *(attached)*, ostatné sú nemanažované *(detached)*.

Ilustrácia metódy equals a manažovaných objektov

Na ilustráciu doplníme projekt o jednoduchú metódu, ktorá:

- Najprv vytvorí nový objekt triedy `Person` a zavolá `persist` na jeho vloženie do databázy.
- Potom použije metódu `find`, ktorá vyhľadá podľa kľúča novovytvorený záznam, načíta a vráti späť ako objekt triedy `Person`.

- Nakoniec overí metódou `equals`, že tieto dva objekty sú ekvivalentné, t. j. reprezentujú ten istý záznam v databáze.

Metódu implementujeme v dvoch verziách:

- **Prvá verzia** použije pre metódy `persist` i `find` ten istý manažér entít. Môžete sa presvedčiť, že metóda `find` nenačíta údaje do nového objektu, ale vráti priamo referenciu na pôvodný objekt vytvorený v prvom kroku. V skutočnosti vôbec údaje z databázy nečíta.
- **Druhá verzia** po volaní `persist` použitý manažér zavrie a na čítanie vytvorí nový. Metóda `find` teraz vráti novú inštanciu. V pamäti budú teda dva rôzne objekty s tým istým kľúčom. Manažovaný, t. j. synchronizovaný s databázou, bude však len druhý.

```
Person p;
p = new Person();
p.setName("Fero");
EntityManagerFactory emf = Persistence.createEntityManagerFactory("vsaPU");

EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
em.persist(p);
em.getTransaction().commit();
long id = p.getId();
System.out.println("id: " + id);
em.close();

em = emf.createEntityManager();
Person p2 = em.find(Person.class, id);
System.out.println("p == p2 :     " + (p == p2));
System.out.println("p.equals(p2): " + p.equals(p2));
```

Doplnenú implementáciu s oboma verziami nájdete v projekte **p21b**.

2.4 Synchronizácia stavu s databázou - Metódy entity managera

Okrem metód `persist` a `find` poskytuje manažér entít radu ďalších metód na synchronizáciu stavu entitného objektu s databázou:

- `detach` – uvoľní manažovaný objekt z množiny objektov spravovaných manažérom entít,
- `clear` – uvoľní všetky objekty spravované manažérom entít,
- `refresh` – obnoví pôvodný stav manažovaného objektu údajmi z databázy,
- `merge` – aktualizuje stav v databáze údajmi z objektu (nemusí byť manažovaný) a vráti manažovaný objekt s aktualizovaným stavom,
- `remove` – odstráni z databázy záznam so stavom entitného objektu. (Samotný objekt sa stane nemanažovaným, existuje však ďalej.)

Ilustráciu použitia týchto metód nájdete v projekte **p21c**.

Projekt obsahuje niekoľko funkcií, ktoré môžeme spúšťať postupne za sebou a používa opäť entitnú triedu `Person`.

2.5 Generovanie entitnej triedy z tabuľky

`Netbeans` IDE poskytuje wizard `Entity Classes from Database …`, ktorým je možné vygenerovať entitnú triedu pre existujúcu tabuľku.

Poznámka: Tento generátor nie je súčasťou JPA implementácie. Podobnú fukcionalitu môžu poskytovať aj iné IDE, resp. nástroje, ich správanie však môže byť odlišné.

Vytvoríme nový projekt a vygenerujeme v ňom entitnú triedu pre tabuľku `PERSON`, s ktorou sme pracovali v predchádzajúcom príklade. Vygenerovaný zdrojový kód ilustruje viacero ďalších nepovinných, ale užitočných JPA anotácií:

- `@Table(name = "PERSON")` anotácia triedy umožňuje zadať názov tabuľky pre entitnú triedu. Využiť ju teda môžeme, ak je meno tabuľky odlišné od mena triedy.
- `@Column(name = "NAME")` anotácia dátového člena umožňuje zadať názov stĺpca tabuľky pre daný dátový člen.
- `@Basic(optional = false)` anotácia hovorí, že dátový člen objektu nesmie mať hodnotu null.
  Poznámka: Táto anotácia však neznamená, že hodnoty v databáze sú povinné a pri generovaní tabuľky z entitnej triedy sa vytvorí stĺpec s vlastnosťou '`notNULL`'. To možno dosiahnuť anotáciou `@Column(nullable=false)`.

2.6 JPQL dotazy

Všimnite si anotácie vygenerovanej triedy `Person`:

```
@Entity
@Table(name = "PERSON")
@NamedQueries({
    @NamedQuery(name = "Person.findAll", query = "SELECT p FROM Person p"),
    @NamedQuery(name = "Person.findById",
                query = "SELECT p FROM Person p WHERE p.id = :id"),
    @NamedQuery(name = "Person.findByBorn",
                query = "SELECT p FROM Person p WHERE p.born = :born"),
    @NamedQuery(name = "Person.findByMarried",
                query = "SELECT p FROM Person p WHERE p.married = :married"),
    @NamedQuery(name = "Person.findByName",
                query = "SELECT p FROM Person p WHERE p.name = :name"),
    @NamedQuery(name = "Person.findBySalary",
                query = "SELECT p FROM Person p WHERE p.salary = :salary")})
public class Person implements Serializable {
```

Anotácie `@NamedQuery` definujú pomenované dopyty pre čítanie údajov z databázy. Pre špecifikáciu dopytu však namiesto SQL používajú jeho *'klon'* jazyk ***JPQL - Java Persistance Query Language***.

Tento jazyk používa namiesto názvov tabuliek a stĺpcov mená entitných tried a ich dátových členov. V nasledujúcich kapitolách uvidíme, že poskytuje aj ďalšiu objektovo orientovanú podporu, napr. dopyty nad hierarchiou entitných tried.

Použitie JPQL-dopytov ilustrujú nasledujúce ukážky:

- Jednoduchý pomenovaný dopyt

```
  TypedQuery<Person> q = em.createNamedQuery("Person.findAll", Person.class);
  List<Person> pl = q.getResultList();
  for (Person p : pl) {
    System.out.println(p.getName() + " narodeny=" + p.getBorn());
}
```

- Dopyt s parametrami

```
  TypedQuery<Person> q1 = em.createNamedQuery("Person.findByName",Person.class);
  q.setParameter("name", "Fero");
```

- Jazyk JPQL môžeme použiť aj na vytvorenie nepomenovaného dopytu metódou `createQuery`.
  Vyhľadanie osôb bez dátumu narodenia:

```
TypedQuery<Person> q2 = em.createQuery(
  "select p from Person p where p.born is null", Person.class);
```

- Dopyt s jedinou návratovou hodnotou – zistenie počtu osôb s platom nižším ako 1000

```
TypedQuery<Long> q3 = em.createQuery(
  "select count(p) from Person p where p.salary < 1000", Long.class);
System.out.println("" + q3.getSingleResult());
```

Pozri projekt **p22jpql**.

2.7 JPA controller

JPA controller je fasáda nad entity managerom, ktorá poskytuje sadu jednoduchých CRUD metód pre prácu s jednou databázovou tabuľkou. Podobné komponenty sú známe aj v mnohých iných prostrediach pod názvom DAO.
JPA controller si pre každú jednu zmenu v tabuľke vytvára vlastnú transakciu, preto jeho použitie nie je vhodné v situáciách, ktoré si vyžadujú rozsiahlejšie transakcie zahŕňajúce zmeny viacerých entitách.
V `netbeans` je možné vygenerovať JPA controller wizardom `JPA Controller Classes from Entity Classes…`

Cvičenia

Cvičenie 2.1 Mapovanie rôznych dátových typov.
Modifikujte projekt **p21a** a pozorujte, ako sa prejavia zmeny vo vygenerovanej tabuľke:

- V aplikácii inicializujte aj dátové členy `married` a `born` objektu `Person`.

Návod pre inicializáciu dátumu:

```
   private static Date toDate(int y, int m, int d) {
       Calendar calendar = Calendar.getInstance();
       calendar.set(y, m, d, 0, 0, 0);
       return calendar.getTime();
   }
```

- Členu `married` dajte anotáciu `@Column(columnDefinition="BOOLEAN")`
- Zmeňte typ člena `born` na `LocalDate` resp. `LocalDateTime` a odstráňte anotáciu `@Temporal`.

Porovnajte správanie pre rôzne typy databáz.

Cvičenie 2.2 Mechanizmus generovania kľúčov.
Databázy môžu poskytovať viacero mechanizmov na generovanie kľúčov. Voľbu mechanizmu umožňuje argument anotácie `@GenerationType`. Vyskúšajte aplikáciu v projekte **p21a** s rôznymi voľbami mechanizmu a pre rôzne databázy.

- Do súboru `persistence.xml` pridajte riadky:

```
<property name="eclipselink.logging.level.sql" value="FINE"/>
<property name="eclipselink.logging.parameters" value="true"/>
```

  Aktivuje sa tým logovanie sql-príkazov generovaných manažérom entít.
- Spustite aplikáciu a v logu pozorujte generované SQL príkazy. Všimnite si príkazy potrebné na generovanie kľúča.
- Zistite, čo sa stane, ak pre tabuľku s generovaným kľúčom zadáme v aplikácii hodnotu kľúčovému dátovému členu. Overte v databáze, či bude mať kľúč zadanú alebo inú, generovanú hodnotu.

Cvičenie 2.3 Použite aplikáciu z projektu **p21a** a porovnajte prácu s generovanými a negenerovanými kľúčmi:

- Stratégiu generovania databázy ponechajte `create`.
- Spustite aplikáciu dvakrát za sebou a presvedčte sa, že v databáze vznikli dva nové záznamy, ktoré sa líšia len hodnotou kľúča.
- V entitnej triede odstráňte anotáciu `@GeneratedValue`.
  Keďže teraz nebude kľúč generovaný databázou, zadajte v aplikácii vhodnú hodnotu aj pre kľúčový dátový člen.

- Spustite opäť aplikáciu dvakrát za sebou (s rovnakým kľúčom). Čo sa stane teraz?

Cvičenie 2.4 Použite projekt **p21b** a sledujte synchronizáciu stavu objektu s databázou.

- Aktivujte logovanie generovaných sql-príkazov.
- Na koniec aplikácie pridajte kód, ktorý zmení plat v oboch objektoch a vykoná transakciu

```
p2.setSalary(3300.0);
em.getTransaction().begin();
p.setSalary(2200.0);
em.getTransaction().commit();
```

- Spustite aplikáciu a overte v logu, či metóda `find` skutočne číta z databázy.
- V databáze príp. v logu overte, ktorý plat sa zapísal do databázy.

Porovnajte obe verzie aplikácie.

Cvičenie 2.5 V projekte **p21c** aktivujte logovanie, spustite aplikáciu a v logu pozorujte SQL-príkazy generované jednotlivými príkazmi.

Cvičenie 2.6 Mapovanie entitnej triedy na existujúcu tabuľku.

V novom projekte vytvorte entitnú triedu `Osoba` s dátovými členmi

```
@Id
private Long id;
private String meno;
private double plat;
private Date narodeny;
private boolean zenaty;
```

- V anotáciách `@Table` a `@Column` nastavte argument `name` tak, aby bola trieda `Osoba` namapovaná na tabuľku `PERSON` z predchádzajúceho cvičenia.
- Stratégiu na generovanie nechajte `none`.
- Implementujte aplikáciu, ktorá vytvorí niekoľko osôb s rôznymi údajmi a vloží ich do databázy.

Cvičenie 2.7

Pokračujte v predchádzajúcom cvičení a implementujte ďalšie aplikácie na čítanie a zápis osôb:
- pomocou JPQL-dotazu načítajte všetky osoby, zvýšte im plat o 100 a zapíše zmenu do databázy.
- pomocou JPQL-dotazu zistite mená osôb, ktoré majú plat nižší ako 1000.0.
- pomocou JPQL-dotazu zistite celkovú sumu platov všetkých osôb.